Model Checking

Administrivia

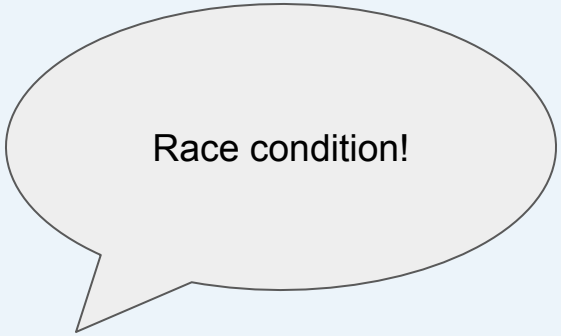
- HW5 due today
- HW6 section tomorrow
- Quiz Friday
- Marcin Guest Lecture Friday

Idea of Model Checking

- Prove that your program never reaches a “bad” state
- Prove that your program always reaches a “good” state

Why model check?

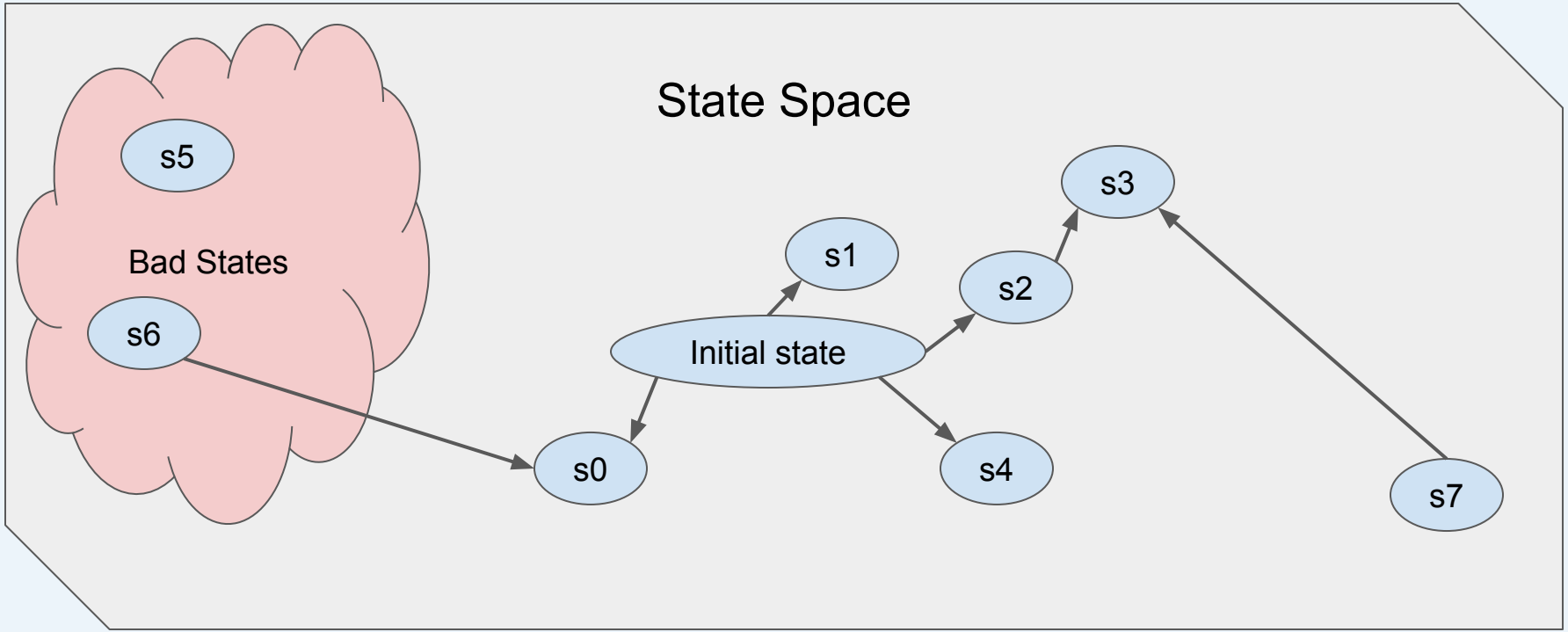
```
let lock = Arc::new(Mutex::new(0u64));
let lock2 = lock.clone();

thread::spawn(move || {
    *lock.lock().unwrap() = 1;
});

assert_eq!(0, *lock2.lock().unwrap());
```



In order to model check, we need:

- Set of states S {s0, s1, s2}
- Transition function T {s1->s0, s2->s0}
- Initial state(s)
- Some proposition of what makes good/bad states

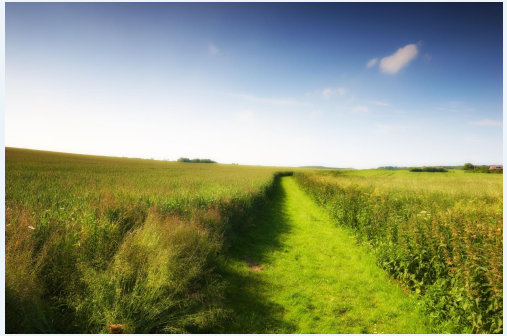
Visual example



Safety and Liveness

- Safety: Something bad will never happen
  - There must be no path to any bad state

- Liveness: Something good will always happen
  - Every path must terminate in a good state
  - Cycles must not exist



CPU 0: a; b; c;
CPU 1: d; e; f;

One example

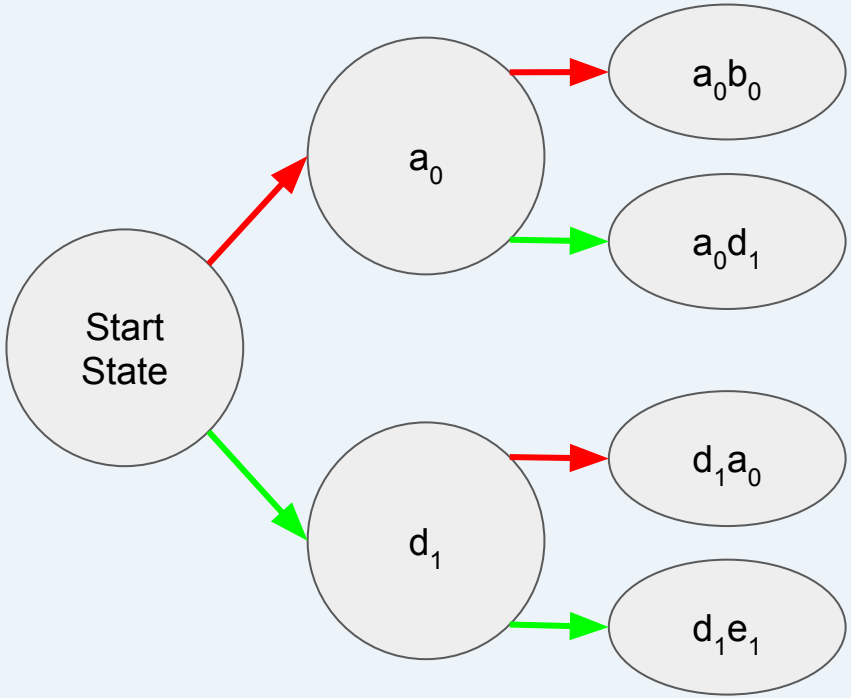
Let’s model core-level interleavings

Every instruction ran on any processor is a new state

Bad states happen when “c” is run without “d” having ran

The most precise model

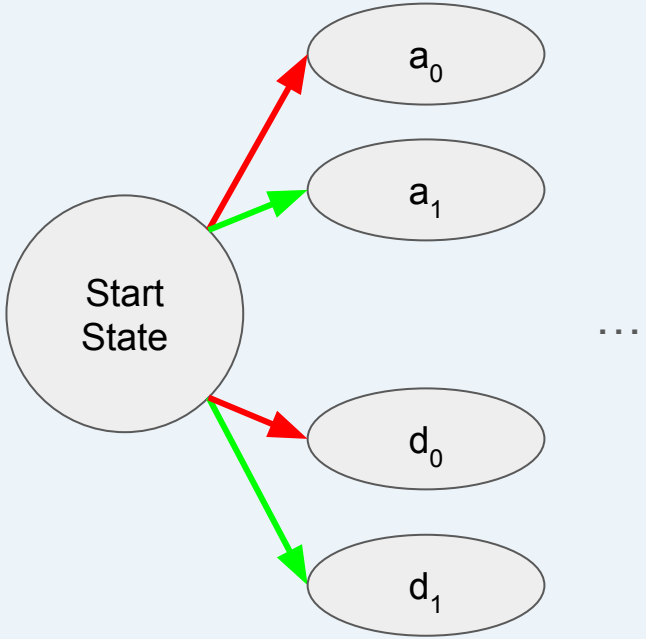
CPU 0: a; b; c;
CPU 1: d; e; f;



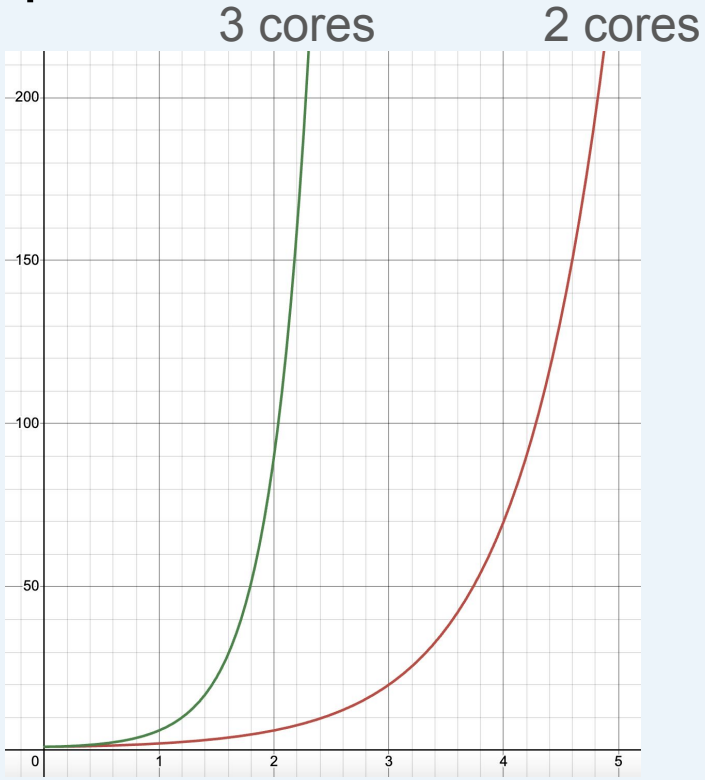
…

The even more precise model

Thread 0: a; b; c;
Thread 1: d; e; f;

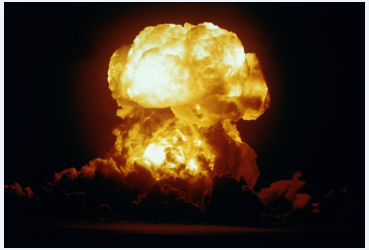


Combinatorial Explosion

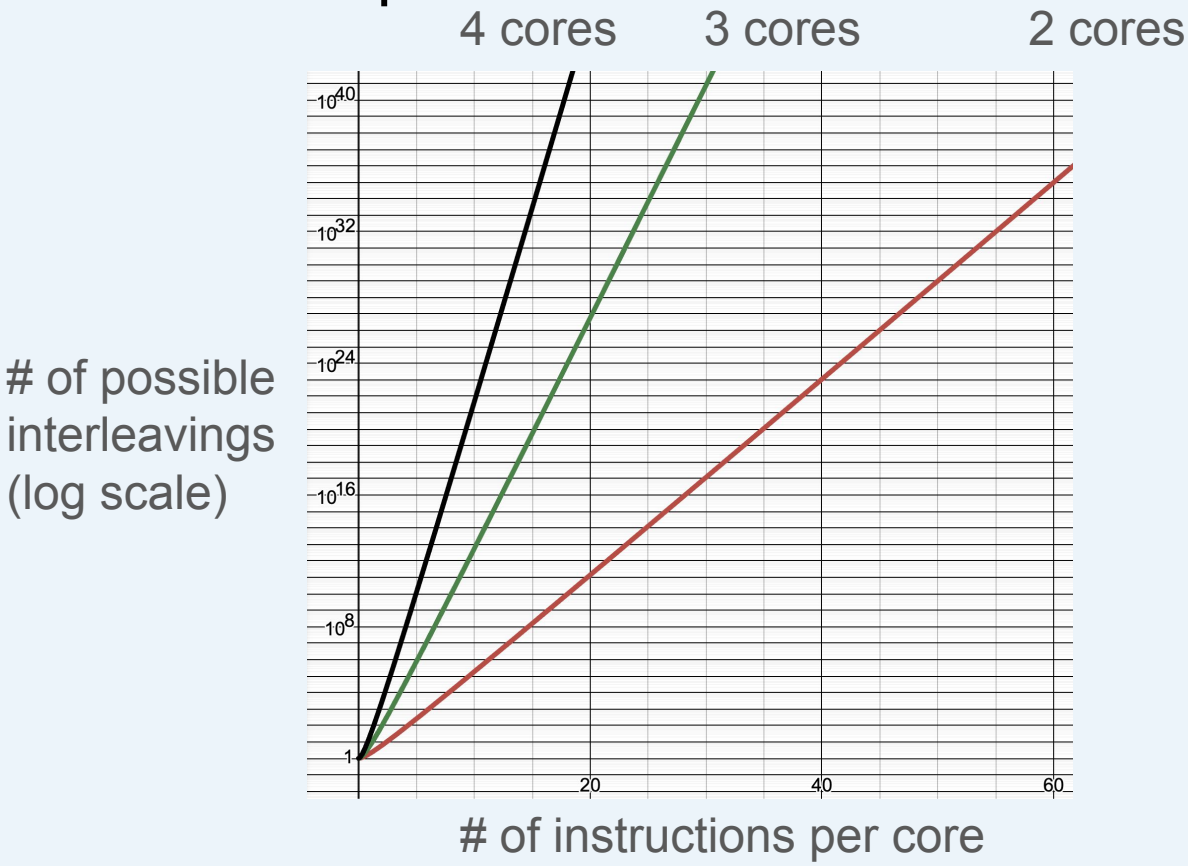


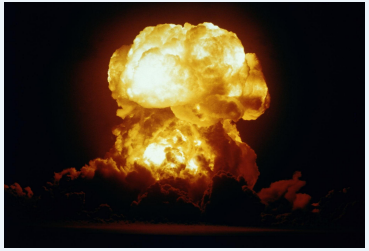
# of possible interleavings

# of instructions per core



Combinatorial Explosion



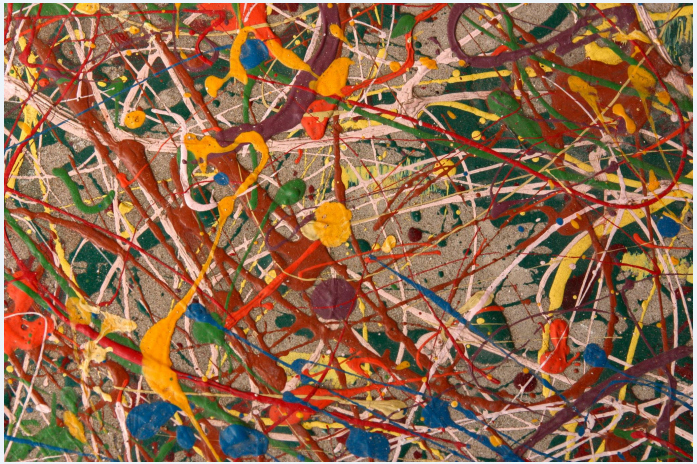


We must abstract!

Perhaps 10^10 states is feasible to brute force, giving us ~20 “instructions/core”

In other words, we can search up to 20 transitions from initial state

Hopefully this is enough to catch bugs. If not, abstract more, or use find some clever tricks…



State Reduction Techniques

- Assumptions
- Symmetry
- Abstraction
- Data independence
- Equivalent state detection
- Rust-specific techniques
- Even more (e.g. partial order reduction)

Assumptions

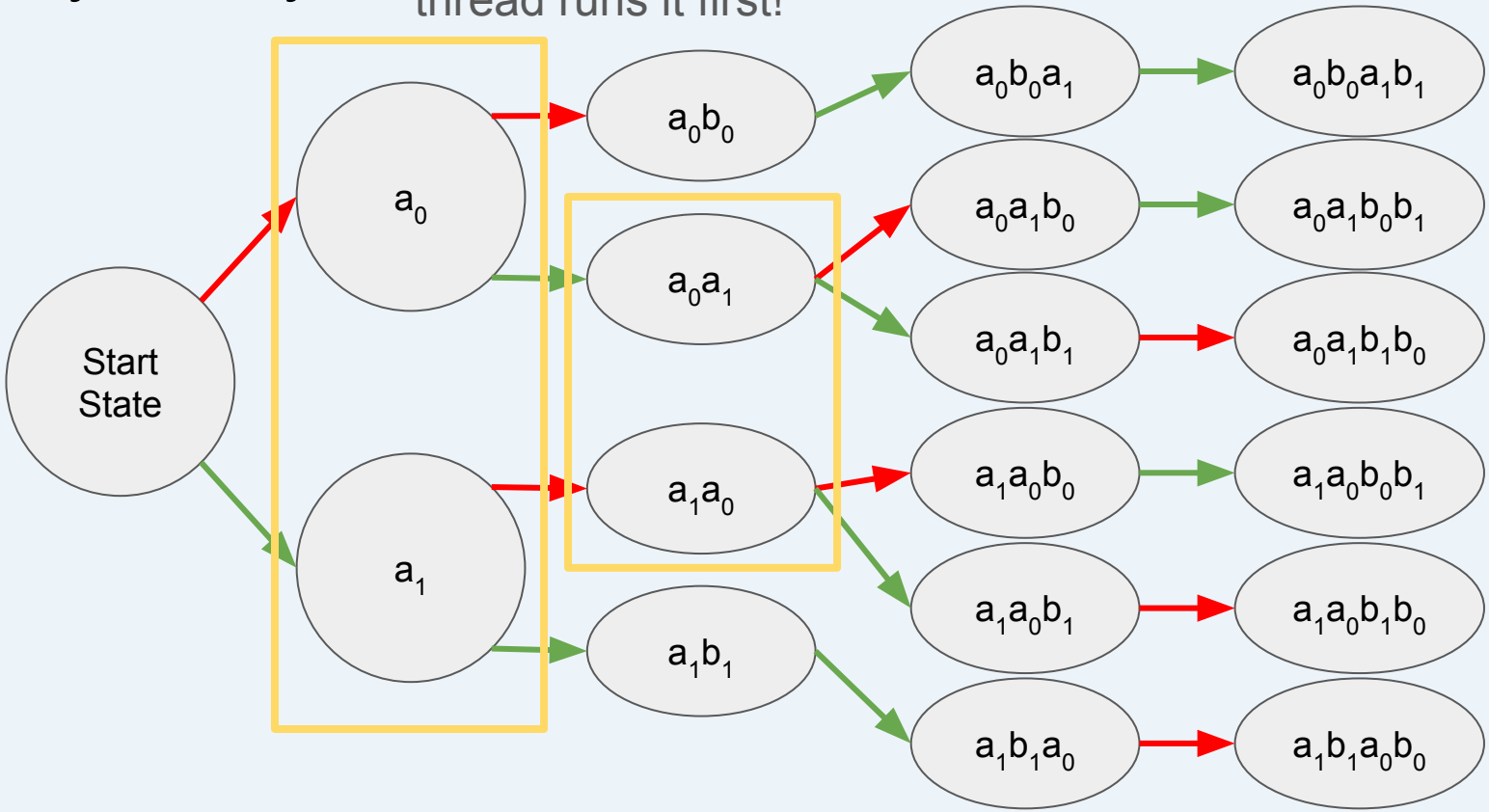
We don’t need to model core-level correctness, only semantic correctness

- Thread-level interleaving is enough, we don’t need to model core-level interleaving on top of that
- We can assume it doesn’t matter which core a thread runs on

Symmetry

If the instructions are the same on each thread, we don't care which thread runs it first!

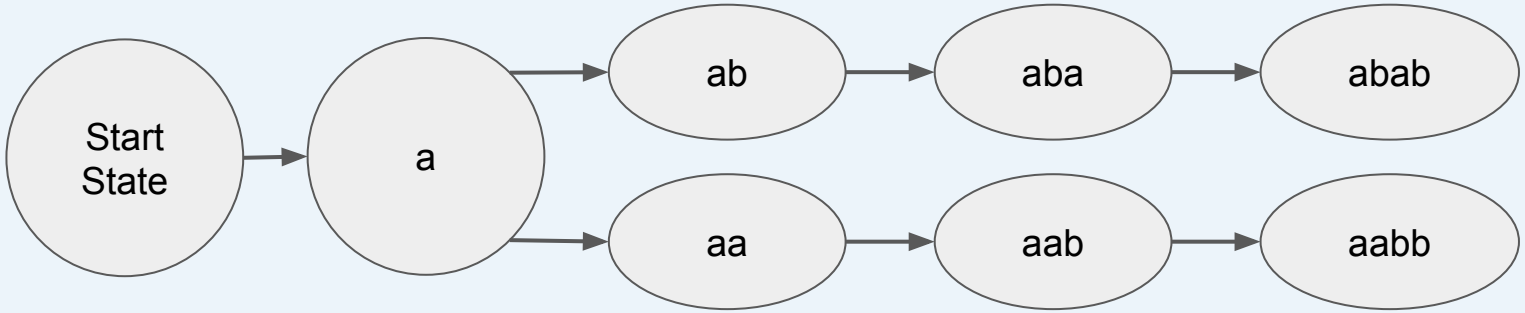
Thread 0: a; b;
Thread 1: a; b;



Symmetry

If the instructions are the same on each thread, we don't care which thread runs it first!

Thread 0: a; b;
Thread 1: a; b;



Abstraction

Rather than model instructions, let’s model memory!

State = an assignment to all variables

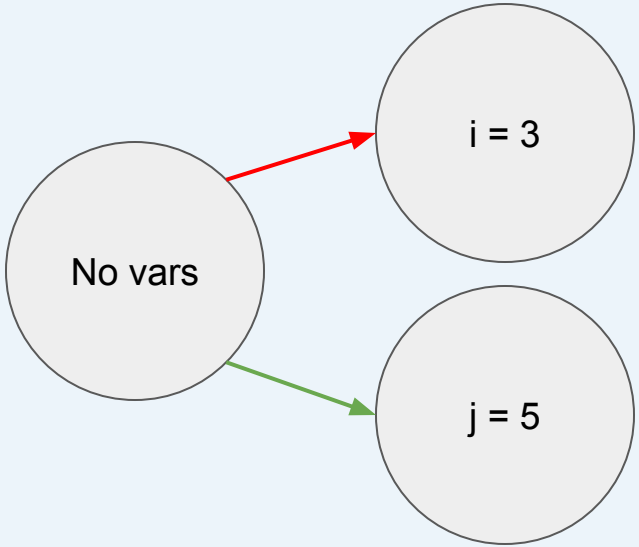
State abstraction

Thread 0:

```
let i = 3;
```

Thread 1:

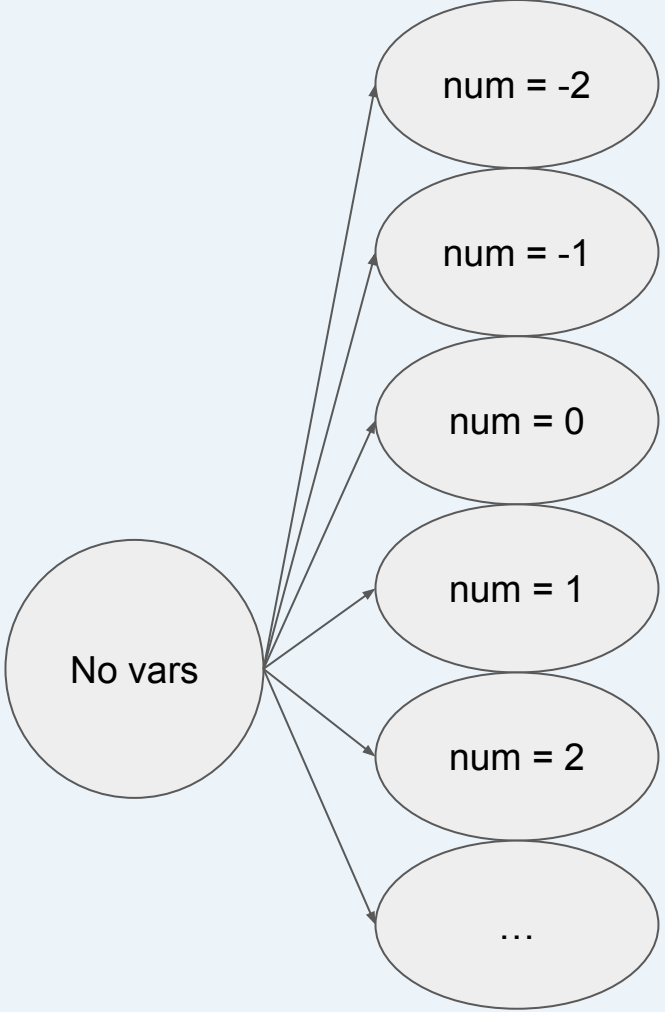
```
let j = 5;
```



Maybe this is too many states… perhaps some variables “don’t matter”

Data Independence

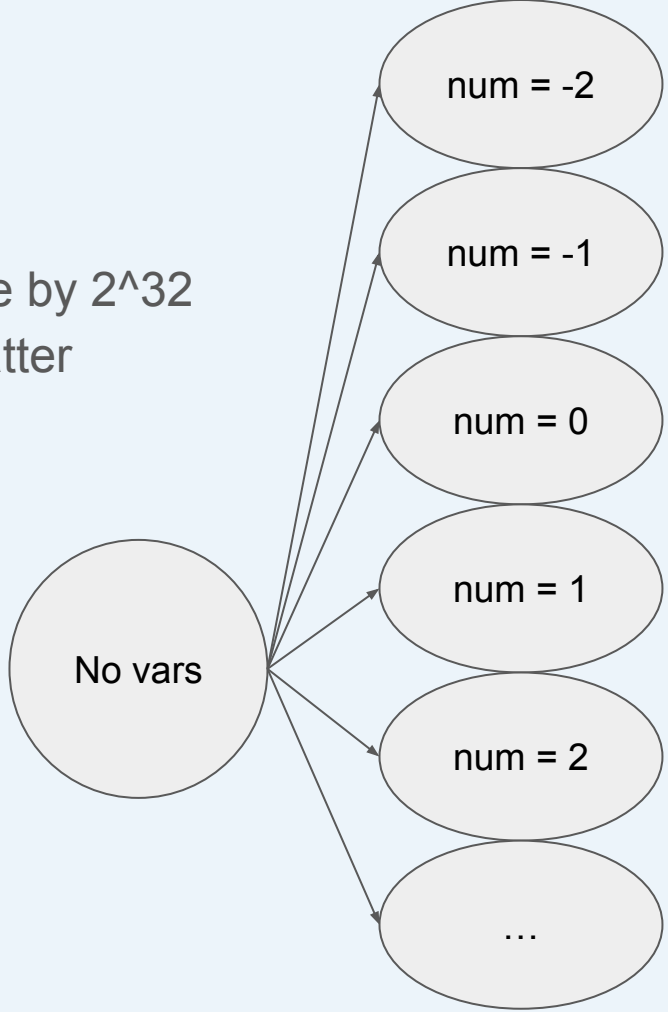
```
let num: i32 = get_user_input();
if (num > 0) {
    goto_new_state();
} else {
    goto_different_state();
}
```



Data Independence

- Every i32 multiplies our state space size by 2^32
- Often times, the exact value doesn't matter

```
let num: i32 = get_user_input();
if (num > 0) {
    goto_new_state();
} else {
    goto_different_state();
}
```



Equivalent State Detection

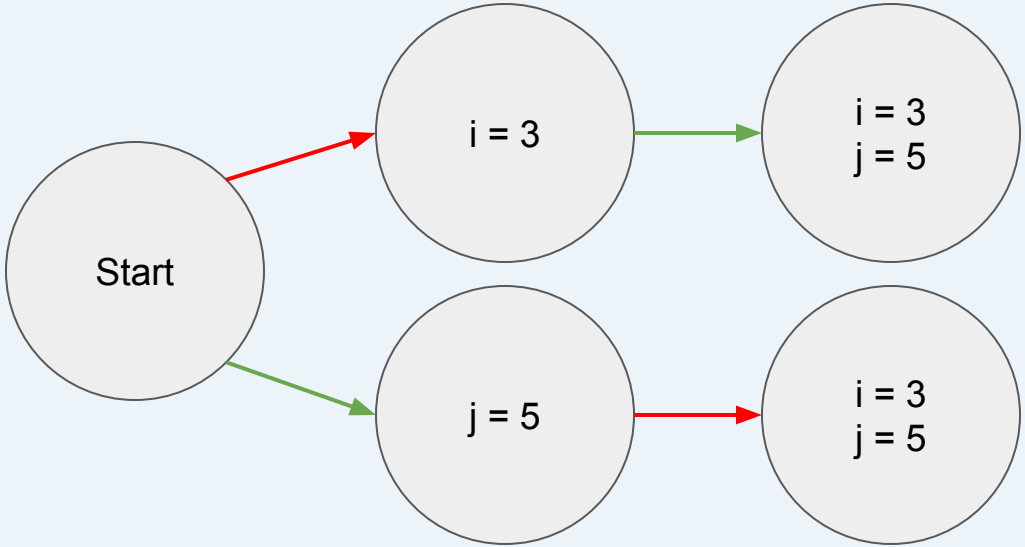
- After nondeterminism, merge identical states into one node

Thread 0:

```
let i = 3;
```

Thread 1:

```
let j = 5;
```



Equivalent State Detection

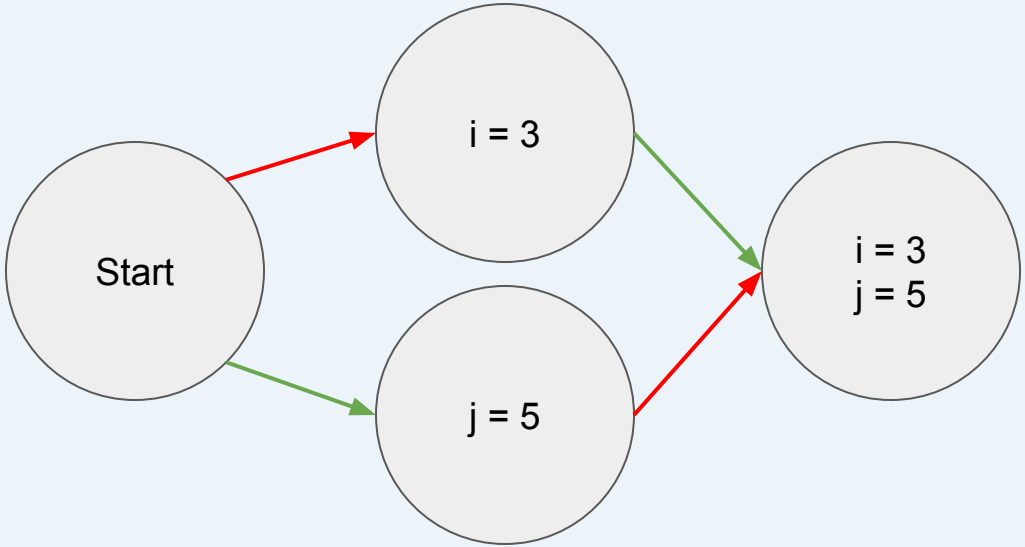
- After nondeterminism, merge identical states into one node

Thread 0:

```
let i = 3;
```

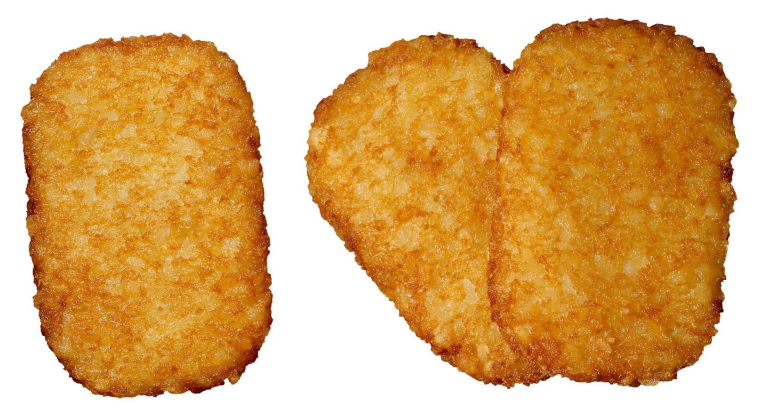
Thread 1:

```
let j = 5;
```



Hashing

- In order to provide equivalent state detection, the state must be hashable
  - Maintain a hash table (state hash -> node)
- Some data types are not easily hashable, e.g. pointers



Rust-specific state reduction

When can threads impact each other?

Only at synchronization primitives!

- Atomics
- Mutex
- Barrier
- …

Rust-specific state reduction

We only transition across threads at synchronization points!

```
Thread 0:

a = 0;
b = 0;
{
    let c = m.lock();
    *c = 0;
}
```

```
Thread 1:

{
    let c = m.lock();
    *c = 1;
}
```

We don’t care about this intermediate state!

Rust-specific state reduction

We only transition across threads at synchronization points!

```
Thread 0:

a = 0;
b = 0;
{
    let c = m.lock();
    *c = 0;
}
```

```
Thread 1:

{
    let c = m.lock();
    *c = 1;
}
```

All we care about is which thread acquires this lock first!

Rust-specific state reduction

We only transition across threads at synchronization points!

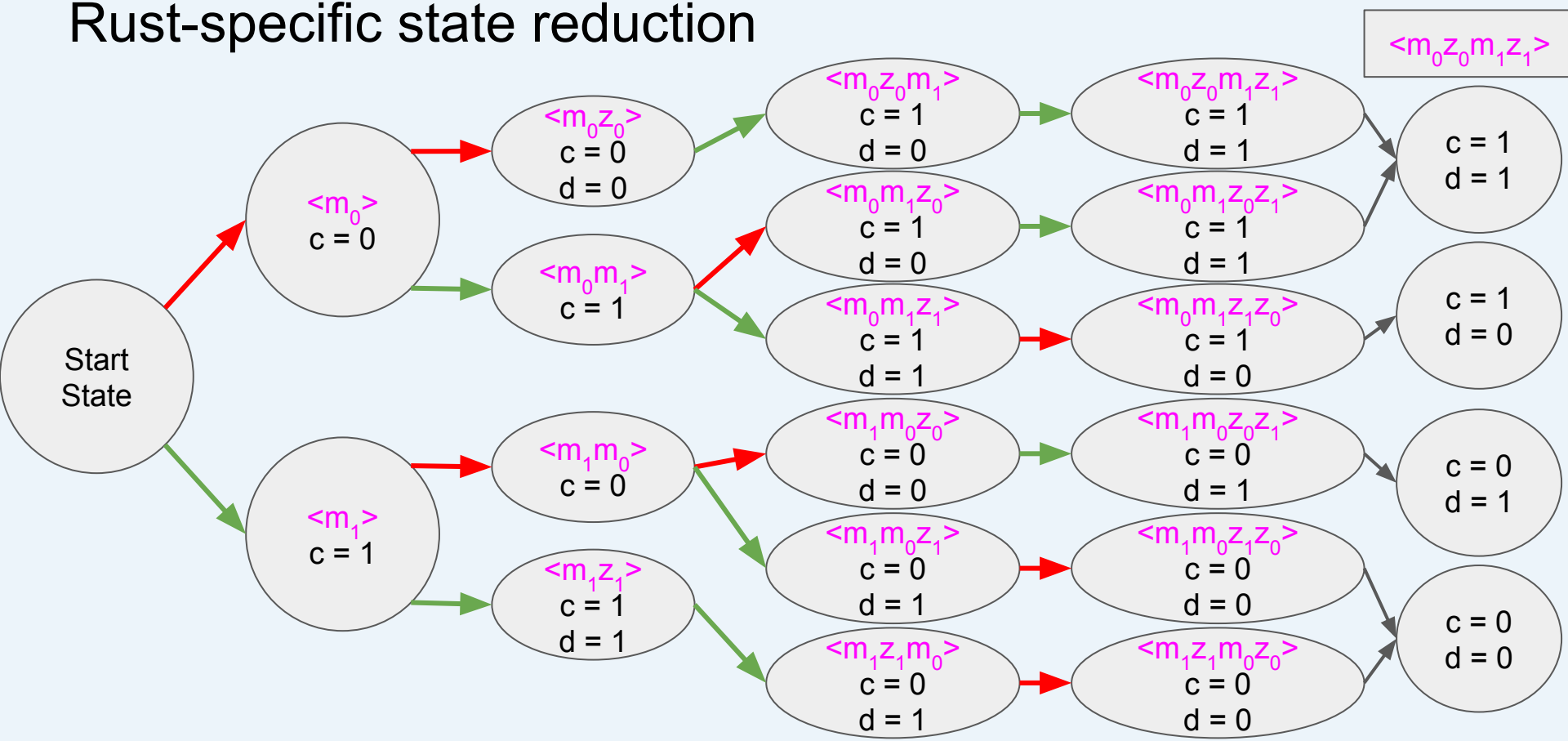
```
Thread 0:
local stuff
{
    let c = m.lock();
    *c = 0;
}
local stuff
{
    let d = z.lock();
    *d = 0;
}
finish
```

```
Thread 1:
local stuff
{
    let c = m.lock();
    *c = 1;
}
local stuff
{
    let d = z.lock();
    *d = 1;
}
finish
```

Rust-specific state reduction

- We define a state s to contain all variables, plus an unordered set of all synchronization operations by each thread
- A transition can only happen for valid paths through the program, but each synchronization primitive can switch to any other thread

Rust-specific state reduction



Race Condition determinism

Nondeterminism may not be bad! The application may be built to handle it.

“Race Condition” just means a different interleaving caused a different outcome.

How do we know when nondeterminism is an issue in practice? Use asserts!

Rust-specific state reduction

```
Thread 0:
local stuff
{
    let c = m.lock();
    *c = 0;
}
local stuff
{
    let d = z.lock();
    *d = 0;
}
finish
```

```
Thread 1:
local stuff
{
    let c = m.lock();
    *c = 1;
}
local stuff
{
    let d = z.lock();
    *d = 1;
}
finish
```

```
assert(c == 1);
```

Finishing the entire model

- Good states are those that do not trigger assertions, and whose paths do not contain cycles (livelock potential!)

- Bad states are the opposite

Loom

- A Rust model checker
- Just replace your synchronization primitives with Loom’s primitives!
- Loom explores the state space by rerunning the entire program, then trying a new interleaving
  - Requires complete control over the scheduler, hence the need to use Loom types

Shuttle

Like Loom, but doesn’t explore *all* states to make even more problems searchable

Bonus: LTL

Linear Temporal Logic: the language to describe assertions over a state automata

Plug ‘n’ chug into a SAT / SMT solver

Bonus Reading: Loom

https://docs.rs/loom/latest/loom/

https://docs.rs/shuttle/0.8.1/shuttle/